

Product Requirements Document: PI Agent Loop

Version: 2.0 (Harness Engineering Edition)

Date: 2026-02-16 **Author:** John Ross + Little John **Status:** Approved for Implementation

1. Executive Summary

PI is an autonomous code generation system that builds complex software projects from a single high-level goal. It decomposes goals into small tasks, executes each through a rigorous **plan** → **write tests** → **code** → **test** → **review** cycle, accumulates institutional knowledge via progressive disclosure, and optionally rebuilds the entire project from scratch using what it learned—producing structurally superior code on the second pass.

The system is powered by a cheap, fast LLM (MiniMax M2.5 Lightning) and designed around the principle that **compute is free but quality is everything**. Rather than optimizing for fewer API calls, it optimizes for code quality through redundancy: strict Test-Driven Development (TDD), multiple independent review passes, throwaway-and-redo cycles, and full-project iterations.

2. Problem Statement

Current AI coding agents operate in one of two modes:

- **Single-shot:** Give it a task, get code back. No iteration, no review, no learning. Quality is a coin flip.
- **Conversation-driven:** Human stays in the loop, reviewing and correcting. Quality improves but doesn't scale—the human becomes the bottleneck.

Neither approach can autonomously build a complex project with consistent quality. The missing ingredients are:

- **Decomposition:** Breaking big goals into right-sized tasks.
- **Mechanical Enforcement:** Hard structural gates that no LLM can talk its way past.
- **The “Homework” Problem:** Preventing the AI from writing shallow tests to conveniently pass its own flawed implementation.
- **Independent Review:** A reviewer that hasn't seen the coding conversation (unbiased).
- **Context Exhaustion:** LLMs drowning in massive, unstructured memory files over long builds.
- **Willingness to Rebuild:** First drafts teach; second drafts ship.

3. Product Vision

“Build it twice. The first time to learn, the second time to ship.”

PI is not a code generator. It’s a **code engineering system** that mirrors how the best human teams work:

- Brainstorm before planning.
- Write failing tests before writing implementation code (TDD).
- Test mechanically and structurally, not by opinion.
- Review with fresh eyes.
- Document what you learned, but query it selectively.
- If the first attempt isn’t great, throw it away and do it properly.

4. Architecture Overview

4.1 Package Structure

PI is a TypeScript monorepo (pnpm workspaces) with strict domain boundaries enforced by dependency-cruiser.

PACKAGE	PURPOSE
@pi/ai	Unified multi-provider LLM API with streaming, TypeBox validation, token tracking.
@pi/agent-core	The brain: agentLoop, projectLoop, prompts, gates, TDD state machine.
@pi/tui	Terminal UI with differential rendering.
@pi/coding-agent	Interactive CLI agent with file tools + structural guards.
@pi/web-ui	Web interface (mini-lit, Tailwind, IndexedDB).
@pi/mom	Slack bot with Docker sandboxing.
@pi/pods	vLLM deployment CLI for GPU providers.

4.2 Core Dependency Graph (Mechanically Enforced)

```
@pi/ai (foundation LLM abstraction)
  ↑
@pi/agent-core (depends on @pi/ai)
  ↑
@pi/coding-agent (depends on @pi/agent-core + @pi/tui)
```

Rule: The above graph is not just a convention; it is **mechanically enforced** via tools like `dependency-cruiser` or `eslint-plugin-boundaries`. If an agent writes code that violates these boundaries, the structural gate will fail the turn.

4.3 LLM Provider

- **Primary:** MiniMax M2.5 Lightning via OpenAI-compatible API at `api.minimaxi.chat/v1`
- The provider adapter (`@pi/ai`) handles: SSE streaming, `<think>` tag stripping, token usage extraction, and cross-provider handoffs.

5. The Two Loops

5.1 Inner Loop: `agentLoop()` — Single Task Execution

Handles one discrete task through a **strict TDD state machine**. Crucially, the testing phase is completely decoupled from the implementation phase to ensure objective verification.

PLAN → WRITE_TESTS → RED_GATE → CODE → GREEN_GATE → PEER_REVIEW → DONE

STATE	WHAT HAPPENS	LLM CALL?	MECHANICAL?
PLAN	Primary LLM creates implementation plan from task.	Yes	No
WRITE_TESTS	Dedicated prompt writes tests based purely on Acceptance Criteria.	Yes	No
RED_GATE	Run tests. They must fail . Proves tests are valid and target unimplemented code.	No	Yes (Hard Gate)
CODE	Primary LLM writes implementation to pass the tests.	Yes	No
GREEN_GATE	Run <code>pnpm run validate</code> (Tests + Lint + Arch check). Must pass.	No	Yes (Hard Gate)
PARSE_ERROR	If tests/lint fail, parse output into structured diagnostics.	No	Yes
SELF_CORRECT	LLM analyzes errors, produces correction patch.	Yes	No
PEER_REVIEW	Secondary Reviewer Agent checks code quality (clean context).	Yes	No
DONE	Task complete.	No	No
FAIL	Exhausted max retries. Mark for throwaway/redo.	No	No

Critical Constraints:

- **Mechanical Gate:** GREEN_GATE runs real commands. An LLM cannot override a failing test suite or boundary linter.

- **Clean Context Review:** The PEER_REVIEW agent receives the task, plan, diff, and learnings. It does NOT get the brainstorming, self-correction history, or coding rationale. This mathematical isolation prevents confirmation bias.

5.2 Outer Loop: `projectLoop()` — Multi-Task Orchestration

Takes a high-level goal and orchestrates the full build:

```
BRAINSTORM → DECOMPOSE → [for each task: agentLoop → GATE → REVIEW → COMPACT_LEARN] → INTEGRATE → VALIDATE → DONE
```

- **Phase 1: BRAINSTORM:** Explore problem space. Propose 2-3 approaches. Apply YAGNI. Output: recommended approach, `ARCHITECTURE.md` boundaries.
- **Phase 2: DECOMPOSE:** Break goal into small tasks (max 20). Every task has explicit acceptance criteria.
- **Phase 3: EXECUTE & PROGRESSIVE DISCLOSURE:**
 - For each task: run `agentLoop(task)`.
 - **Routing Decision:**
 - `approved` → commit + learn + next task
 - `needs_fixes` + redo available → THROW AWAY (`git checkout .`) + redo with learnings.
 - `needs_fixes` + no redo → self-correct patch.
 - `rejected` → fail task, attempt recovery.
- **Compound/Learn (Progressive Disclosure):** Instead of dumping a massive `LEARNINGS.md` file into the prompt, the system runs a `COMPACT_LEARNINGS` prompt every 5 tasks to deduplicate and index knowledge. Agents are provided a `query_learnings("topic")` tool to retrieve specific context to avoid prompt exhaustion.
- **Phase 4 & 5: INTEGRATE & VALIDATE:** Full workspace validation (`pnpm run validate`). Final holistic LLM review of the entire project.

6. Advanced Features

6.1 Throw Away and Redo with Learnings

The key insight: The first attempt at code teaches the agent the problem space. Throwing it away and starting fresh produces structurally cleaner code than patching it 15 times.

Flow when review says `needs_fixes` :

1. Log everything learned (what was wrong, what worked, types discovered).
2. `git checkout .` — throw away ALL code from this attempt.
3. Run `agentLoop()` AGAIN for the same task.

4. Second attempt gets original task + plan + “learnings from attempt 1”. It starts with a clean code slate.

6.2 The Brooks Loop (Full-Project Iterations)

“Plan to throw one away; you will, anyhow.” — Fred Brooks

Config: `--iterations 2` (“build twice” mode).

- **Iteration 1 (Reconnaissance Build):** Run full project loop. Goal is learning, not shipping. Write comprehensive `ITERATION_1_LEARNINGS.md` . `git tag iteration-1-complete` . `git checkout` . (delete all code, keep docs).
- **Iteration 2 (Informed Build):** Fresh Brainstorm and Decompose, now armed with complete structural knowledge of the problem, edge cases, and proper task boundaries. Execute tasks. Ship Iteration 2.

6.3 Deterministic Mechanical Gates

Gates are hard validation points. No LLM can talk its way past a failing gate.

GATE	WHEN	WHAT IT CHECKS
<code>validateDecomposition</code>	After DECOMPOSE	Plan is parseable, all tasks have Acceptance Criteria.
<code>redTestGate</code>	Inner Loop (Before Code)	<code>pnpm test</code> fails. Prevents writing tautological tests.
<code>validateWorkspace</code>	After each task	<code>pnpm run validate</code> (lint + typecheck + tests). Exit code 0.
<code>validateArchitecture</code>	Part of <code>validateWorkspace</code>	<code>dependency-cruiser</code> checks against <code>ARCHITECTURE.md</code> . Prevents domain/dependency boundary violations.
<code>validateReview</code>	After REVIEW	Output strictly maps to <code>approved</code> / <code>needs_fixes</code> / <code>rejected</code> .

7. State Management (Git-Bound State)

All state is tracked in files committed to git. The system does not rely on conversation history for memory.

- `PROJECT_STATUS.md` : Live execution plan with `[x]` checkmarks and attempt trackers.
- `LEARNINGS.md` : Accumulated institutional knowledge (compacted & queried via tool).
- `ITERATION_N_LEARNINGS.md` : Per-iteration summaries.
- `ARCHITECTURE.md` : Hard architectural boundaries used by structural linters.
- `EXECUTION_PLAN.md` : The deterministic plan.

Git Operations:

- After successful task: `git add -A && git commit -m "feat(project): <task>"`
 - Before redo: `git checkout .` (throw away code, keep docs)
 - Before iteration 2: `git tag iteration-1-complete && git checkout .`
 - Push is manual.
-

8. CLI Interface

```
# Single Task Runner
npx tsx packages/agent-core/src/runner.ts "Add a hello() function..."

# Project Runner (Brooks mode: build twice)
npx tsx packages/agent-core/src/project-runner.ts --iterations 2 "Build a REST API"

# Environment Variables
MINIMAX_API_KEY=sk-...
PI_TEST_COMMAND="pnpm run validate"
PI_MAX_TASKS=20
PI_MAX_REDO_ROUNDS=1
PI_ITERATIONS=1
```

9. Quality Philosophy

- **The Inverted 80/20 Rule:** Traditional coding is 20% planning, 80% coding. PI is 80% planning, TDD, and review, and 20% coding. Typing the solution is the easy part.
 - **Mechanical Over Magical:** Type errors come from `tsc`. Boundary violations are caught by `dependency-cruiser`, not prompted away.
 - **Fresh Eyes Over Familiar Eyes:** The reviewer never sees the coding conversation. Confirmation bias is the enemy of code quality.
 - **Throw Away Over Patch:** Self-correction accumulates technical debt. Redo-with-learnings produces structurally pristine architecture.
 - **Fully Autonomous:** No human in the loop. The determinism of the gates means the system throws away and retries until it natively passes.
-

10. File Structure Additions

(In addition to standard monorepo scaffolding)

- `packages/agent-core/src/gates.ts` (Now includes `validateArchitecture` and `redTestGate`)
 - `packages/agent-core/src/tdd-agent.ts` (Dedicated test generation logic)
 - `packages/agent-core/src/memory.ts` (Progressive disclosure & learnings compaction)
 - `ARCHITECTURE.md` (System map for dependency-cruiser rules)
 - `AGENTS.md` (Progressive disclosure map for future agents navigating the repo)
-

11. Acceptance Criteria

- [] `agentLoop()` successfully executes single tasks using the TDD flow (WRITE_TESTS → RED_GATE → CODE → GREEN_GATE).
 - [] **TDD Gate:** The system generates a test suite that must fail before implementation begins.
 - [] **Architecture Gate:** Mechanical gates (`dependency-cruiser`) successfully block agents from violating `ARCHITECTURE.md` .
 - [] Peer Reviews operate with zero knowledge of the coding phase (Clean Context).
 - [] Failed reviews successfully trigger a `git checkout` . (Throwaway & Redo) while preserving learnings.
 - [] Memory is managed via `COMPACT_LEARNINGS` (Progressive Disclosure) to prevent context window bloat.
 - [] The Brooks Loop (`--iterations 2`) successfully builds, documents, deletes, and rebuilds the project.
 - [] State is entirely Git-bound and frequently committed.
-

12. The Self-Rebuild Test

The ultimate validation: feed this PRD to the system and ask it to rebuild itself.

```
npx tsx packages/agent-core/src/project-runner.ts --iterations 2 \  
"Implement the PI Agent Loop system as described in docs/PRD.md. Read the PRD  
first, then build everything it specifies."
```

If the output matches or exceeds the hand-built version, the system works.

13. Future: Real-Time Observability Dashboard (v2.1)

The `projectLoop` already emits structured `ProjectLoopEvent` objects for every state transition — brainstorm, decompose, plan, execute, review, fail, retry, learn, integrate, validate. This event stream is the foundation for a real-time observability UI.

13.1 Transport Layer

Events are written as NDJSON to `events.jsonl` in the workspace. A lightweight HTTP server (part of `@pi/web-ui`) serves:

- **SSE endpoint** (`/events`) — streams events to the browser in real-time.
- **REST endpoint** (`/state`) — returns current snapshot (tasks, status, learnings).
- **Static files** — the dashboard SPA.

13.2 Dashboard Views

VIEW	DESCRIPTION
Kanban Board	Tasks flow through columns: Backlog → Planning → Executing → Review → Done / Failed. Cards show retry count, quality score, and time spent.
Timeline	Chronological event stream with phase icons. Click to expand details (plan text, diff, review rationale).
File Explorer	Live view of workspace files. Click a task to see its <code>git diff</code> .
Task Detail	Full context for a single task: spec, plan, code diff, test output, review decision, learnings captured.
Iteration Progress	Progress bar per iteration. Retry burn-down chart. Completed vs remaining tasks.
Token & Cost Tracker	Cumulative token usage and estimated cost from the AI gateway's usage events.
Learnings Feed	Live feed of captured learnings, filterable by category (pattern, anti-pattern, architecture, edge case).
Brooks Loop View	Side-by-side comparison of Iteration 1 vs Iteration 2: task decomposition, code diff, quality scores.

13.3 Implementation Notes

- Single-page app served from `@pi/web-ui`. No build step — vanilla HTML/JS/CSS or mini-lit.
- The dashboard is read-only and passive — it observes the event stream but never interferes with execution.
- Works with both live runs (SSE) and completed runs (load `events.jsonl` from disk).
- The ultimate demo: watching PI rebuild itself in real-time on the dashboard.

“Write one to throw away.” — Fred Brooks, 1975 “Now do it automatically.” — PI, 2026